Activity_Course 3 Automatidata project lab

October 25, 2025

1 Course 3 Automatidata project

Course 3 - Go Beyond the Numbers: Translate Data into Insights

You are the newest data professional in a fictional data consulting firm: Automatidata. The team is still early into the project, having only just completed an initial plan of action and some early Python coding work.

Luana Rodriquez, the senior data analyst at Automatidata, is pleased with the work you have already completed and requests your assistance with some EDA and data visualization work for the New York City Taxi and Limousine Commission project (New York City TLC) to get a general understanding of what taxi ridership looks like. The management team is asking for a Python notebook showing data structuring and cleaning, as well as any matplotlib/seaborn visualizations plotted to help understand the data. At the very least, include a box plot of the ride durations and some time series plots, like a breakdown by quarter or month.

Additionally, the management team has recently asked all EDA to include Tableau visualizations. For this taxi data, create a Tableau dashboard showing a New York City map of taxi/limo trips by month. Make sure it is easy to understand to someone who isn't data savvy, and remember that the assistant director at the New York City TLC is a person with visual impairments.

A notebook was structured and prepared to help you in this project. Please complete the following questions.

2 Course 3 End-of-course project: Exploratory data analysis

In this activity, you will examine data provided and prepare it for analysis. You will also design a professional data visualization that tells a story, and will help data-driven decisions for business needs.

Please note that the Tableau visualization activity is optional, and will not affect your completion of the course. Completing the Tableau activity will help you practice planning out and plotting a data visualization based on a specific business need. The structure of this activity is designed to emulate the proposals you will likely be assigned in your career as a data professional. Completing this activity will help prepare you for those career moments.

The purpose of this project is to conduct exploratory data analysis on a provided data set. Your mission is to continue the investigation you began in C2 and perform further EDA on this data with the aim of learning more about the variables.

The goal is to clean data set and create a visualization.
This activity has 4 parts:

Part 1: Imports, links, and loading

Part 2: Data Exploration * Data cleaning

Part 3: Building visualizations

Part 4: Evaluate and share results

Follow the instructions and answer the questions below to complete the activity. Then, you will complete an Executive Summary using the questions listed on the PACE Strategy Document.

Be sure to complete this activity before moving on. The next course item will provide you with a completed exemplar to compare to your own work.

3 Visualize a story in Tableau and Python

4 PACE stages

- `[Plan](#scrollTo=psz51YkZVwtN&line=3&uniqifier=1)`
- `[Analyze](#scrollTo=mA7Mz_SnI8km&line=4&uniqifier=1)`
- `[Construct](#scrollTo=Lca9c8XON8lc&line=2&uniqifier=1)`
- `[Execute](#scrollTo=401PgchTPr4E&line=2&uniqifier=1)`

Throughout these project notebooks, you'll see references to the problem-solving framework PACE. The following notebook components are labeled with the respective PACE stage: Plan, Analyze, Construct, and Execute.

4.1 PACE: Plan

In this stage, consider the following questions where applicable to complete your code response: 1. Identify any outliers:

- What methods are best for identifying outliers?
- How do you make the decision to keep or exclude outliers from any future models?

At this stage, I will use simple and effective methods to identify outliers: - **Box plots** to visualize values that fall far outside the normal range. - **Descriptive statistics** like mean, median, and standard deviation to notice any unusual differences in the data.

The decision depends on what the outlier represents: - If an outlier seems to come from a **data error or incorrect entry**, I will remove or correct it.

- If the outlier looks like a **real and meaningful event**, I will keep it, because it might show an important pattern or business insight.

- I will also check how much the outliers **influence the results or visualizations** before deciding to remove them.

4.1.1 Task 1. Imports, links, and loading

Go to Tableau Public The following link will help you complete this activity. Keep Tableau Public open as you proceed to the next steps.

Link to supporting materials: Tableau Public: https://public.tableau.com/s/

For EDA of the data, import the data and packages that would be most helpful, such as pandas, numpy and matplotlib.

> **Note:** Although the Tableau activity is optional, I included it to enrich the analysis and accessibility.

[423]:
```
# Import packages and libraries

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import datetime as dt
import seaborn as sns
```

Note: As shown in this cell, the dataset has been automatically loaded in for you. You do not need to download the .csv file, or provide more code, in order to access the dataset and proceed with this lab. Please continue with this activity by completing the following instructions.

[424]:
```
# Load dataset into dataframe
df = pd.read_csv('2017_Yellow_Taxi_Trip_Data.csv')
```

4.2 PACE: Analyze

Consider the questions in your PACE Strategy Document to reflect on the Analyze stage.

4.2.1 Task 2a. Data exploration and cleaning

Decide which columns are applicable

The first step is to assess your data. Check the Data Source page on Tableau Public to get a sense of the size, shape and makeup of the data set. Then answer these questions to yourself:

Given our scenario, which data columns are most applicable? Which data columns can I eliminate, knowing they won't solve our problem scenario?

Consider functions that help you understand and structure the data.

- head()
- describe()

- info()
- groupby()
- sortby()

1- What do you do about missing data (if any)?

2- Are there data outliers? What are they and how might you handle them?

3- What do the distributions of your variables tell you about the question you're asking or the problem you're trying to solve?

1- After checking the dataset with the `info()` function, there are no missing values. All columns show the same number of non-null entries. Since there is no missing data, no cleaning or imputation is needed at this stage.

If missing values were found, possible actions would include:

- Dropping rows or columns with too many missing values.
- Filling missing data with mean, median, or mode values, depending on the variable type.

2- Yes, some potential outliers appear in the numeric data:

- Negative values in fare-related fields (e.g., `fare_amount`, `extra`, `mta_tax`, `improvement_surcharge`, `total_amount`) likely reflect data errors or reversals; flag, review source, and filter from EDA unless justified.
- **Very large fare or Tip amounts** (e.g., over $1,000) may also be outliers that could distort analysis.

Possible actions:

- Investigate the source of negative or extreme values.
- Remove or correct data entries that are not possible in real taxi trips.
- For valid but rare high fares, consider keeping them but note their influence when building visualizations or models.

3-The `trip_distance` and `fare_amount` distributions are **right-skewed**, meaning most trips are short and inexpensive, while a few are very long or costly.

The `passenger_count` data shows that most trips carry **1 passenger**, indicating typical single-rider behavior.

These distributions help identify **normal patterns** in NYC taxi trips and highlight **unusual rides** that may need attention or separate analysis.

Start by discovering, using head and size.

[425]:
```
# shows the first rows of the dataframe
df.head(10)
```

[425]:
```
   Unnamed: 0  VendorID    tpep_pickup_datetime   tpep_dropoff_datetime  \
0    24870114         2   03/25/2017 8:55:43 AM   03/25/2017 9:09:47 AM
1    35634249         1   04/11/2017 2:53:28 PM   04/11/2017 3:19:58 PM
```

```
2   106203690         1   12/15/2017 7:26:56 AM   12/15/2017 7:34:08 AM
3    38942136         2   05/07/2017 1:17:59 PM   05/07/2017 1:48:14 PM
4    30841670         2  04/15/2017 11:32:20 PM  04/15/2017 11:49:03 PM
5    23345809         2   03/25/2017 8:34:11 PM   03/25/2017 8:42:11 PM
6    37660487         2   05/03/2017 7:04:09 PM   05/03/2017 8:03:47 PM
7    69059411         2   08/15/2017 5:41:06 PM   08/15/2017 6:03:05 PM
8     8433159         2   02/04/2017 4:17:07 PM   02/04/2017 4:29:14 PM
9    95294817         1   11/10/2017 3:20:29 PM   11/10/2017 3:40:55 PM

   passenger_count  trip_distance  RatecodeID store_and_fwd_flag  \
0                6           3.34           1                  N
1                1           1.80           1                  N
2                1           1.00           1                  N
3                1           3.70           1                  N
4                1           4.37           1                  N
5                6           2.30           1                  N
6                1          12.83           1                  N
7                1           2.98           1                  N
8                1           1.20           1                  N
9                1           1.60           1                  N

   PULocationID  DOLocationID  payment_type  fare_amount  extra  mta_tax  \
0           100           231             1         13.0    0.0      0.5
1           186            43             1         16.0    0.0      0.5
2           262           236             1          6.5    0.0      0.5
3           188            97             1         20.5    0.0      0.5
4             4           112             2         16.5    0.5      0.5
5           161           236             1          9.0    0.5      0.5
6            79           241             1         47.5    1.0      0.5
7           237           114             1         16.0    1.0      0.5
8           234           249             2          9.0    0.0      0.5
9           239           237             1         13.0    0.0      0.5

   tip_amount  tolls_amount  improvement_surcharge  total_amount
0        2.76           0.0                    0.3         16.56
1        4.00           0.0                    0.3         20.80
2        1.45           0.0                    0.3          8.75
3        6.39           0.0                    0.3         27.69
4        0.00           0.0                    0.3         17.80
5        2.06           0.0                    0.3         12.36
6        9.86           0.0                    0.3         59.16
7        1.78           0.0                    0.3         19.58
8        0.00           0.0                    0.3          9.80
9        2.75           0.0                    0.3         16.55
```

[426]:
```
# Check the total number of elements
df.size
```

[426]: 408582

[427]:
```
# displays descriptive statistics (mean, median, standard deviation, etc.)
df.describe()
```

[427]:
```
         Unnamed: 0      VendorID  passenger_count  trip_distance  \
count  2.269900e+04  22699.000000     22699.000000   22699.000000
mean   5.675849e+07      1.556236         1.642319       2.913313
std    3.274493e+07      0.496838         1.285231       3.653171
min    1.212700e+04      1.000000         0.000000       0.000000
25%    2.852056e+07      1.000000         1.000000       0.990000
50%    5.673150e+07      2.000000         1.000000       1.610000
75%    8.537452e+07      2.000000         2.000000       3.060000
max    1.134863e+08      2.000000         6.000000      33.960000

         RatecodeID  PULocationID  DOLocationID  payment_type   fare_amount  \
count  22699.000000  22699.000000  22699.000000  22699.000000  22699.000000
mean       1.043394    162.412353    161.527997      1.336887     13.026629
std        0.708391     66.633373     70.139691      0.496211     13.243791
min        1.000000      1.000000      1.000000      1.000000   -120.000000
25%        1.000000    114.000000    112.000000      1.000000      6.500000
50%        1.000000    162.000000    162.000000      1.000000      9.500000
75%        1.000000    233.000000    233.000000      2.000000     14.500000
max       99.000000    265.000000    265.000000      4.000000    999.990000

              extra       mta_tax    tip_amount  tolls_amount  \
count  22699.000000  22699.000000  22699.000000  22699.000000
mean       0.333275      0.497445      1.835781      0.312542
std        0.463097      0.039465      2.800626      1.399212
min       -1.000000     -0.500000      0.000000      0.000000
25%        0.000000      0.500000      0.000000      0.000000
50%        0.000000      0.500000      1.350000      0.000000
75%        0.500000      0.500000      2.450000      0.000000
max        4.500000      0.500000    200.000000     19.100000

       improvement_surcharge  total_amount
count           22699.000000  22699.000000
mean                0.299551     16.310502
std                 0.015673     16.097295
min                -0.300000   -120.300000
25%                 0.300000      8.750000
50%                 0.300000     11.800000
75%                 0.300000     17.800000
max                 0.300000   1200.290000
```

Note: `RatecodeID` shows unexpected values (e.g., max=99). NYC TLC rate codes are typically 1–6; flag these as data-quality outliers and filter them out during EDA unless

a valid explanation exists.

[428]:
```
# shows information about columns, data types, and missing values
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22699 entries, 0 to 22698
Data columns (total 18 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   Unnamed: 0             22699 non-null  int64
 1   VendorID               22699 non-null  int64
 2   tpep_pickup_datetime   22699 non-null  object
 3   tpep_dropoff_datetime  22699 non-null  object
 4   passenger_count        22699 non-null  int64
 5   trip_distance          22699 non-null  float64
 6   RatecodeID             22699 non-null  int64
 7   store_and_fwd_flag     22699 non-null  object
 8   PULocationID           22699 non-null  int64
 9   DOLocationID           22699 non-null  int64
 10  payment_type           22699 non-null  int64
 11  fare_amount            22699 non-null  float64
 12  extra                  22699 non-null  float64
 13  mta_tax                22699 non-null  float64
 14  tip_amount             22699 non-null  float64
 15  tolls_amount           22699 non-null  float64
 16  improvement_surcharge  22699 non-null  float64
 17  total_amount           22699 non-null  float64
dtypes: float64(8), int64(7), object(3)
memory usage: 3.1+ MB
```

4.2.2 Task 2b. Assess whether dimensions and measures are correct

On the data source page in Tableau, double check the data types for the applicable columns you selected on the previous step. Pay close attention to the dimensions and measures to assure they are correct.

In Python, consider the data types of the columns. *Consider:* Do they make sense?

Review the link provided in the previous activity instructions to create the required Tableau visualization.

4.2.3 Task 2c. Select visualization type(s)

Select data visualization types that will help you understand and explain the data.

Now that you know which data columns you'll use, it is time to decide which data visualization makes the most sense for EDA of the TLC dataset. What type of data visualization(s) would be

most helpful?

- Line graph
- Bar chart
- Box plot
- Histogram
- Heat map
- Scatter plot
- A geographic map

The columns `tpep_pickup_datetime` and `tpep_dropoff_datetime` were originally stored as object (text) data types.

For this EDA, I will use histograms, box plots, and scatter plots to explore the distributions and relationships between key variables. A line graph and a geographic map will also be useful to show trends over time and the locations of taxi trips.

4.3 PACE: Construct

Consider the questions in your PACE Strategy Document to reflect on the Construct stage.

4.3.1 Task 3. Data visualization

You've assessed your data, and decided on which data variables are most applicable. It's time to plot your visualization(s)!

4.3.2 Boxplots

Perform a check for outliers on relevant columns such as Trip distance and trip duration. Remember, some of the best ways to identify the presence of outliers in data are box plots and histograms.

Note: Remember to convert your date columns to datetime in order to derive total trip duration.

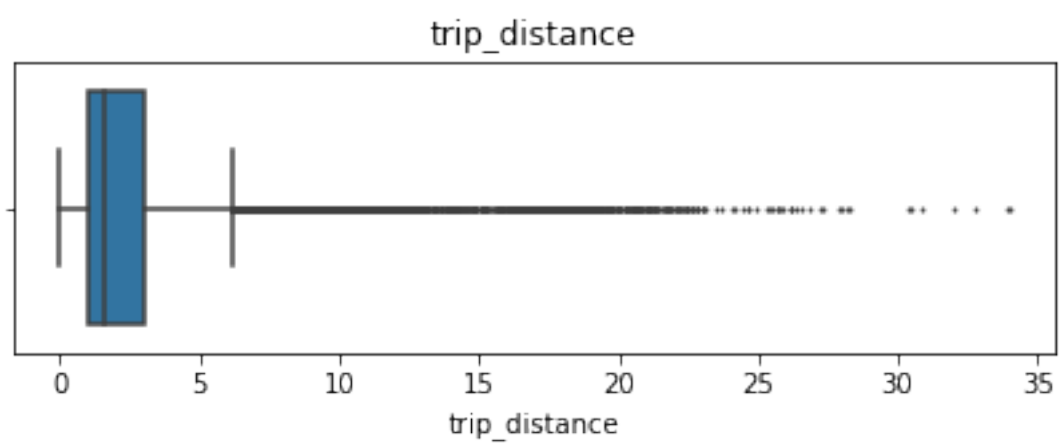
[429]:
```
# Convert data columns to datetime
# The 'tpep_pickup_datetime' and 'tpep_dropoff_datetime' columns are stored as␣
↪objects (text).
# They should be converted to datetime format for proper time-based analysis.
df['tpep_pickup_datetime'] = pd.to_datetime(df['tpep_pickup_datetime'])
df['tpep_dropoff_datetime'] = pd.to_datetime(df['tpep_dropoff_datetime'])
# Derive trip duration in minutes
df['trip_duration_minutes'] = (df['tpep_dropoff_datetime'] -␣
↪df['tpep_pickup_datetime']).dt.total_seconds() / 60
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22699 entries, 0 to 22698
Data columns (total 19 columns):
 #   Column                 Non-Null Count  Dtype
```

```
---   ------                  --------------  -----
 0    Unnamed: 0              22699 non-null  int64
 1    VendorID                22699 non-null  int64
 2    tpep_pickup_datetime    22699 non-null  datetime64[ns]
 3    tpep_dropoff_datetime   22699 non-null  datetime64[ns]
 4    passenger_count         22699 non-null  int64
 5    trip_distance           22699 non-null  float64
 6    RatecodeID              22699 non-null  int64
 7    store_and_fwd_flag      22699 non-null  object
 8    PULocationID            22699 non-null  int64
 9    DOLocationID            22699 non-null  int64
 10   payment_type            22699 non-null  int64
 11   fare_amount             22699 non-null  float64
 12   extra                   22699 non-null  float64
 13   mta_tax                 22699 non-null  float64
 14   tip_amount              22699 non-null  float64
 15   tolls_amount            22699 non-null  float64
 16   improvement_surcharge   22699 non-null  float64
 17   total_amount            22699 non-null  float64
 18   trip_duration_minutes   22699 non-null  float64
dtypes: datetime64[ns](2), float64(9), int64(7), object(1)
memory usage: 3.3+ MB
```

Trip distance

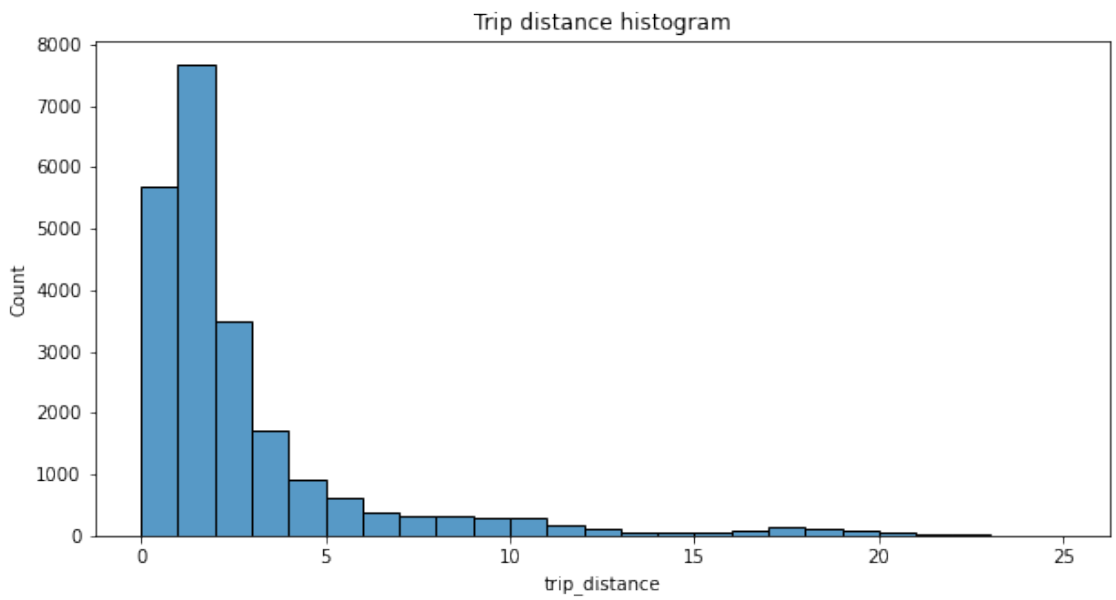
[430]:
```
# Create box plot of trip_distance
plt.figure(figsize=(7,2))
plt.title('trip_distance')
sns.boxplot(data=None, x=df['trip_distance'], fliersize=1);
```



The boxplot of Trip distance shows that most trips are relatively short, with 50% of the distances falling between approximately 1 miles and 3 miles. The median trip distance is around 1.6 miles, indicating that half of the trips are shorter than this value. However, the data are right-skewed,

meaning there are several unusually long trips extending up to about 34 miles. These outliers significantly increase the mean distance compared to the median. Overall, the majority of trips are short, but a few long trips create a long right tail in the distribution.

[431]:
```
# Create histogram of trip_distance
plt.figure(figsize=(10,5))
sns.histplot(df['trip_distance'], bins=range(0,26,1))
plt.title('Trip distance histogram');
```

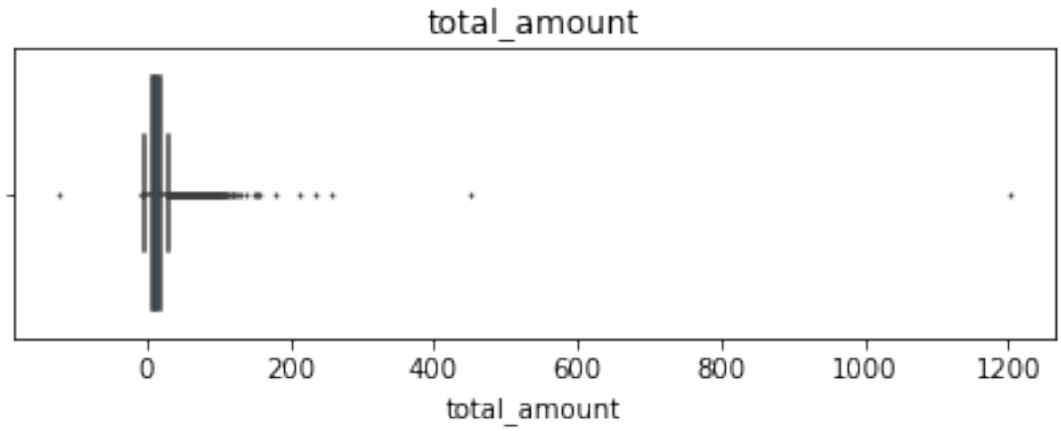


The histogram shows most trips between 0–3 miles, with a right-skewed tail toward longer distances. The frequency decreases sharply as the distance increases, indicating that longer trips are much less common. A few trips extend beyond 20 miles, which suggests the presence of outliers or exceptional long-distance rides.

Overall, the distribution is right-skewed (positively skewed), meaning most trips are clustered on the left side of the histogram, while a long tail stretches to the right. This pattern implies that short-distance trips are the most frequent and represent the typical behavior in the dataset, while long-distance trips occur only occasionally.

Total amount

[432]:
```
# Create box plot of total_amount
plt.figure(figsize=(7,2))
plt.title('total_amount')
sns.boxplot(x=df['total_amount'], fliersize=1);
```

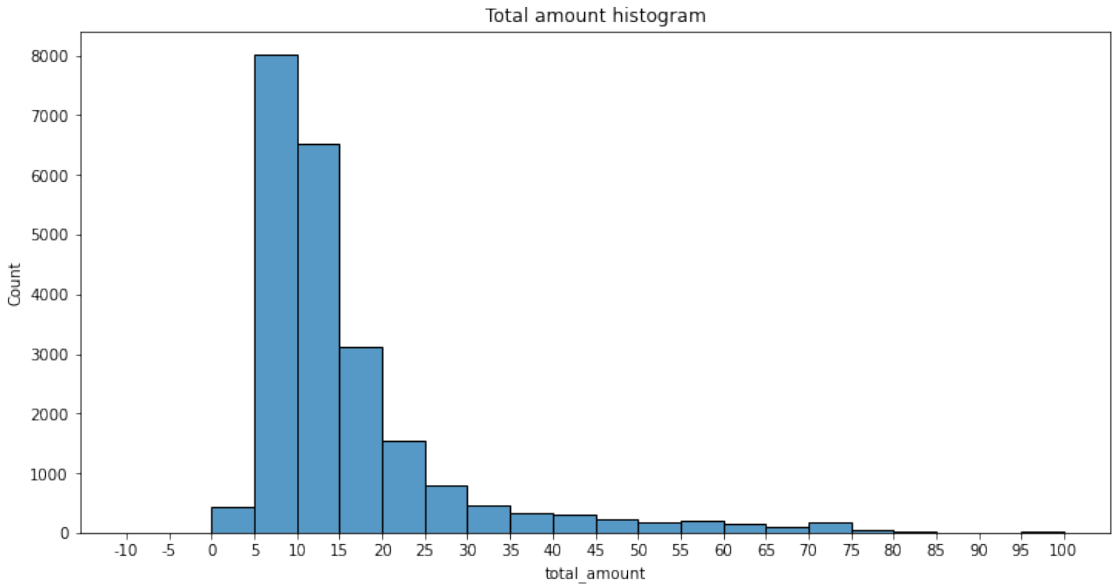


The boxplot of `total_amount` shows that most trips have relatively low total fares, with **50%** of the values ranging between approximately **$8.75** and **$17.80**.
The median fare is about **$11.80**, while the mean is slightly higher (**$16.31**), indicating a right-skewed distribution caused by a few expensive trips.

There are several extreme outliers, with some trips reaching Total amounts above **$1,200.00**. Additionally, a few negative values (around **-$120**) appear, which may represent data entry errors or refunds.

Overall, the data suggest that most trips have moderate costs, but a small number of unusually high fares significantly increase the average.

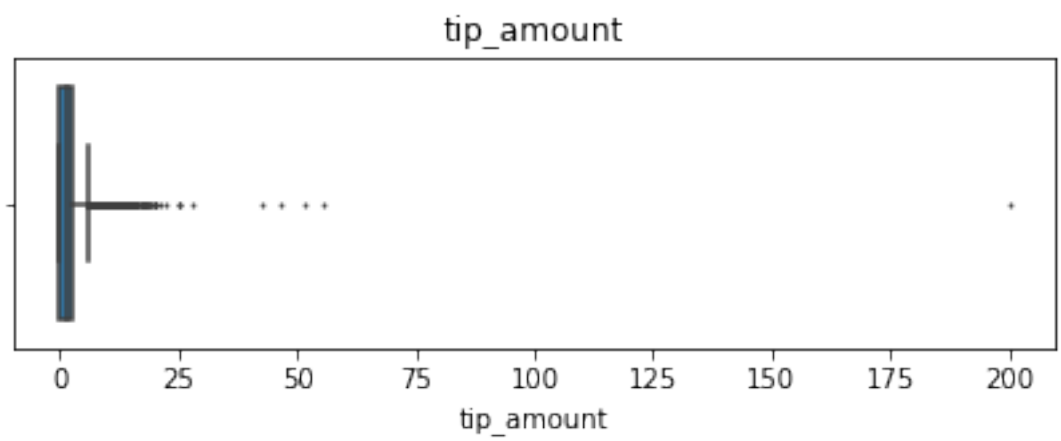
[433]:
```
# Create histogram of total_amount
plt.figure(figsize=(12,6))
ax = sns.histplot(df['total_amount'], bins=range(-10,101,5))
ax.set_xticks(range(-10,101,5))
ax.set_xticklabels(range(-10,101,5))
plt.title('Total amount histogram');
```



The total cost of each trip also has a distribution that skews right, with most costs falling in the $5-15 range.

Tip amount

[434]:
```
# Create box plot of tip_amount
plt.figure(figsize=(7,2))
plt.title('tip_amount')
sns.boxplot(x=df['tip_amount'], fliersize=1);
```

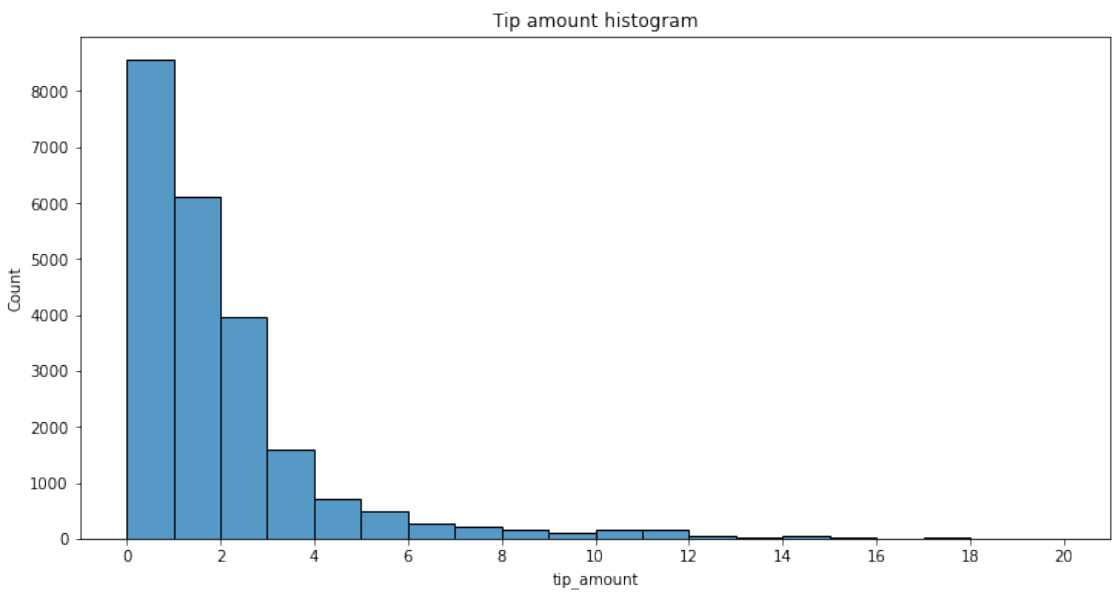


The boxplot of `tip_amount` shows that most tips are relatively small, while a few extreme outliers significantly increase the overall average.
The median tip is **$1.35**, with **25%** of trips having no tip (**$0.00**) and **75%** of tips below approx-

imately **$2.45**.
The mean tip is slightly higher (**$1.83**), reflecting the influence of a few very large tips.

A maximum value of **$200.00** is identified as an extreme outlier, which may represent a data entry error or a rare, exceptionally generous payment.
Overall, the data indicate that most customers give small or no tips, while only a few provide unusually high ones.

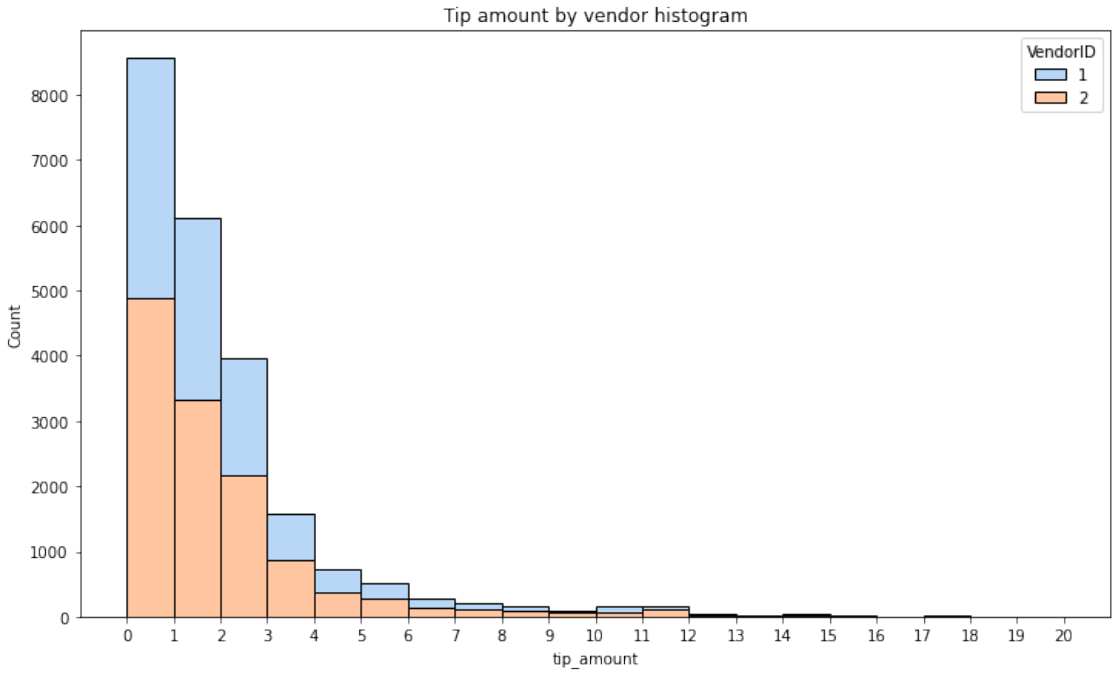
[435]:
```
# Create histogram of tip_amount
plt.figure(figsize=(12,6))
ax = sns.histplot(df['tip_amount'], bins=range(0,21,1))
ax.set_xticks(range(0,21,2))
ax.set_xticklabels(range(0,21,2))
plt.title('Tip amount histogram');
```



The distribution for tip amount is right-skewed, with nearly all the tips in the $0-3 range.

tip_amount by vendor

[436]:
```
# Create histogram of tip_amount by vendor
plt.figure(figsize=(12,7))
ax = sns.histplot(data=df, x='tip_amount', bins=range(0,21,1),
                  hue='VendorID',
                  multiple='stack',
                  palette='pastel')
ax.set_xticks(range(0,21,1))
ax.set_xticklabels(range(0,21,1))
plt.title('Tip amount by vendor histogram');
```

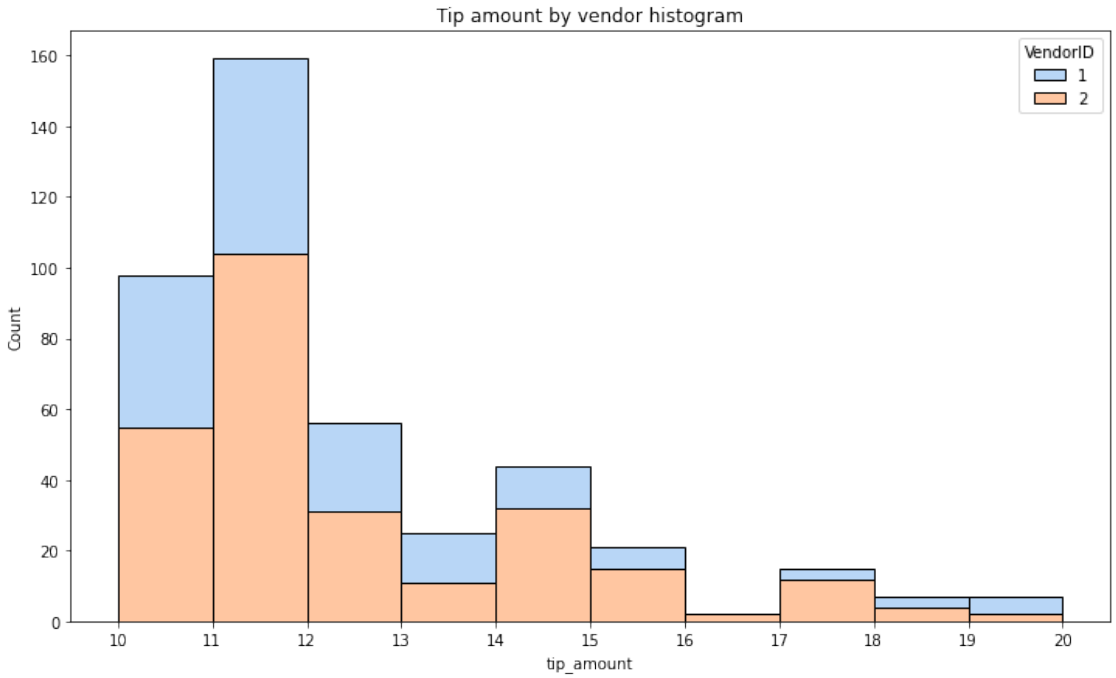


Separating the tip amount by vendor reveals that there are no noticeable aberrations in the distribution of tips between the two vendors in the dataset. Vendor two has a slightly higher share of the rides, and this proportion is approximately maintained for all tip amounts.

Next, zoom in on the upper end of the range of tips to check whether vendor one gets noticeably more of the most generous tips.

Next, zoom in on the upper end of the range of tips to check whether vendor one gets noticeably more of the most generous tips.

[437]:
```
# Create histogram of tip_amount by vendor for tips > $10
tips_over_ten = df[df['tip_amount'] > 10]
plt.figure(figsize=(12,7))
ax = sns.histplot(data=tips_over_ten, x='tip_amount', bins=range(10,21,1),
                  hue='VendorID',
                  multiple='stack',
                  palette='pastel')
ax.set_xticks(range(10,21,1))
ax.set_xticklabels(range(10,21,1))
plt.title('Tip amount by vendor histogram');
```



The proportions are maintained even at these higher tip amounts, with the exception being at highest extremity, but this is not noteworthy due to the low sample size at these tip amounts.

Mean tips by passenger count

Examine the unique values in the `passenger_count` column.

[438]:
```
#The unique() function returns all the distinct values in a column, removing␣
↪any duplicates.
#It helps identify which different values appear in that column.

df['passenger_count'].value_counts()
```

[438]:
```
1    16117
2     3305
5     1143
3      953
6      693
4      455
0       33
Name: passenger_count, dtype: int64
```

Nearly two thirds of the rides were single occupancy, though there were still nearly 700 rides with as many as six passengers. Also, there are 33 rides with an occupancy count of zero, which doesn't make sense. These would likely be dropped unless a reasonable explanation can be found for them.

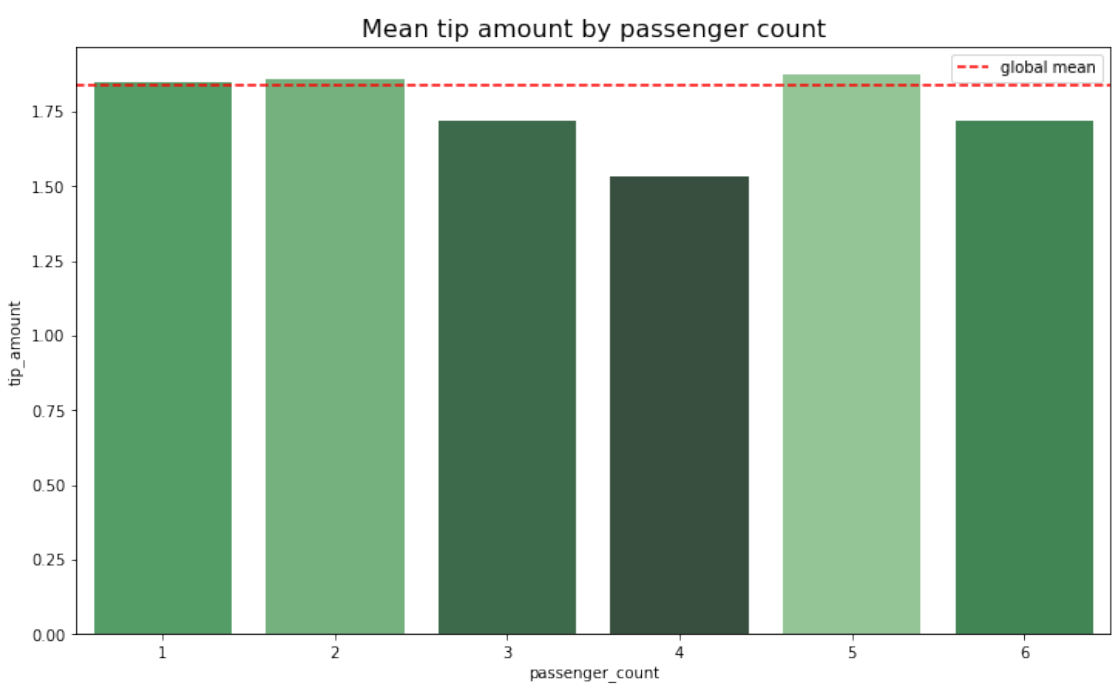
[439]:
```
# Calculate mean tips by passenger_count
mean_tips_by_passenger_count = df.groupby(['passenger_count']).
 ↪mean()[['tip_amount']]
mean_tips_by_passenger_count
```

[439]:

passenger_count	tip_amount
0	2.135758
1	1.848920
2	1.856378
3	1.716768
4	1.530264
5	1.873185
6	1.720260

[440]:
```
# Create bar plot for mean tips by passenger count

data = mean_tips_by_passenger_count.tail(-1)
pal = sns.color_palette("Greens_d", len(data))
rank = data['tip_amount'].argsort().argsort()
plt.figure(figsize=(12,7))
ax = sns.barplot(x=data.index,
            y=data['tip_amount'],
            palette=np.array(pal[::-1])[rank])
ax.axhline(df['tip_amount'].mean(), ls='--', color='red', label='global mean')
ax.legend()
plt.title('Mean tip amount by passenger count', fontsize=16);
```



Mean tip amount varies very little by passenger count. Although it does drop noticeably for four-passenger rides, it's expected that there would be a higher degree of fluctuation because rides with four passengers were the least plentiful in the dataset (aside from rides with zero passengers).

Create month and day columns

[441]:
```
# Create a month column
df['month'] = df['tpep_pickup_datetime'].dt.month_name()
# Create a day column
df['day'] = df['tpep_pickup_datetime'].dt.day_name()
```

> Note: This code standardizes the time-related columns (day of week and month) to ensure they appear in the correct chronological order and use short, clear abbreviations for cleaner and more consistent visualizations.

Plot total ride count by month

Begin by calculating total ride count by month.

[442]:
```
# Get total number of rides for each month
monthly_rides = df['month'].value_counts()
monthly_rides
```

[442]:
```
March        2049
October      2027
April        2019
May          2013
January      1997
June         1964
December     1863
November     1843
February     1769
September    1734
August       1724
July         1697
Name: month, dtype: int64
```

The months are out of order.

Reorder the results to put the months in calendar order.

[444]:
```
# Reorder the monthly ride list so months go in order
month_order = ['January', 'February', 'March', 'April', 'May', 'June', 'July',
         'August', 'September', 'October', 'November', 'December']

monthly_rides = monthly_rides.reindex(index=month_order)
monthly_rides
```

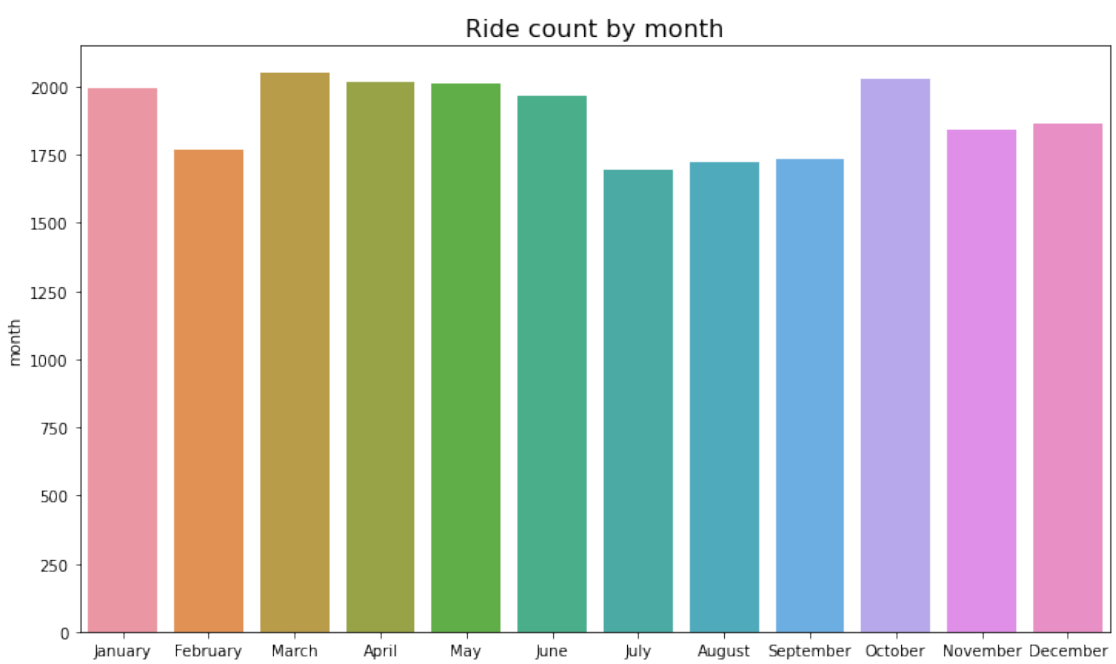
[444]:
```
January      1997
February     1769
March        2049
April        2019
May          2013
June         1964
July         1697
August       1724
September    1734
October      2027
November     1843
December     1863
Name: month, dtype: int64
```

[446]:
```
# Show the index
monthly_rides.index
```

[446]:
```
Index(['January', 'February', 'March', 'April', 'May', 'June', 'July',
       'August', 'September', 'October', 'November', 'December'],
      dtype='object')
```

[447]:
```
# Create a bar plot of total rides per month

plt.figure(figsize=(12,7))
ax = sns.barplot(x=monthly_rides.index, y=monthly_rides)
ax.set_xticklabels(month_order)
plt.title('Ride count by month', fontsize=16);
```



Monthly rides are fairly consistent, with notable dips in the summer months of July, August, and September, and also in February.

Plot total ride count by day

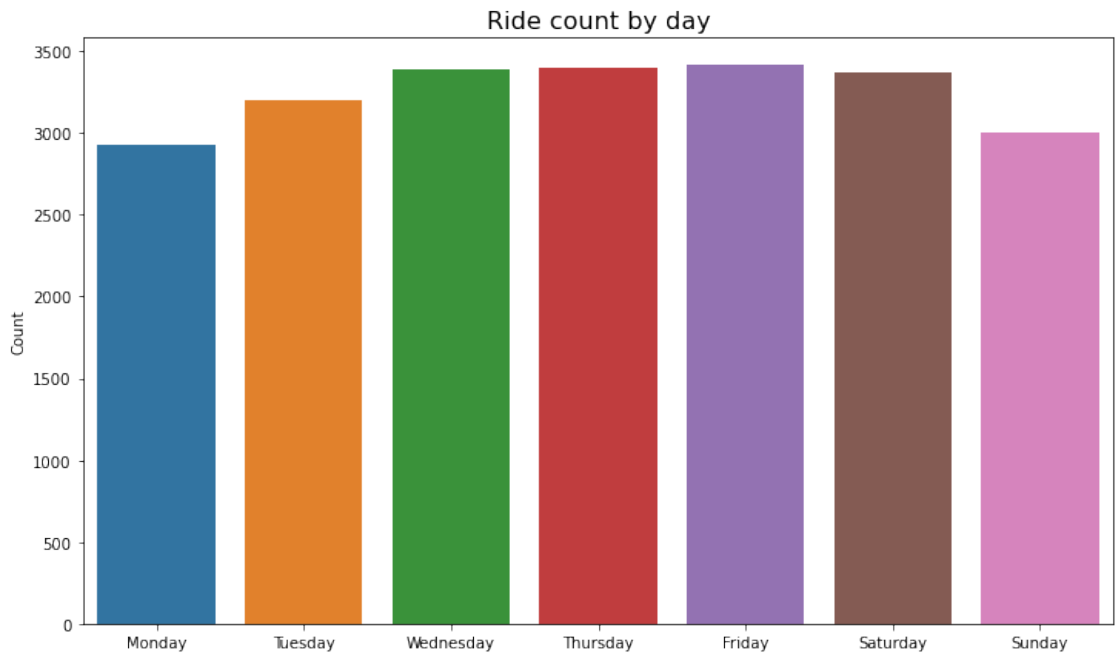
Repeat the above process, but now calculate the total rides by day of the week.

[448]:
```
# Repeat the above process, this time for rides by day
daily_rides = df['day'].value_counts()
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday',␣
 ↪'Saturday', 'Sunday']
daily_rides = daily_rides.reindex(index=day_order)
daily_rides
```

[448]:
```
Monday       2931
Tuesday      3198
Wednesday    3390
Thursday     3402
Friday       3413
Saturday     3367
Sunday       2998
Name: day, dtype: int64
```

[449]:
```
# Create bar plot for ride count by day

plt.figure(figsize=(12,7))
ax = sns.barplot(x=daily_rides.index, y=daily_rides)
ax.set_xticklabels(day_order)
ax.set_ylabel('Count')
plt.title('Ride count by day', fontsize=16);
```



Suprisingly, Wednesday through Saturday had the highest number of daily rides, while Sunday and Monday had the least.

Plot total revenue by day of the week

Repeat the above process, but now calculate the total revenue by day of the week.

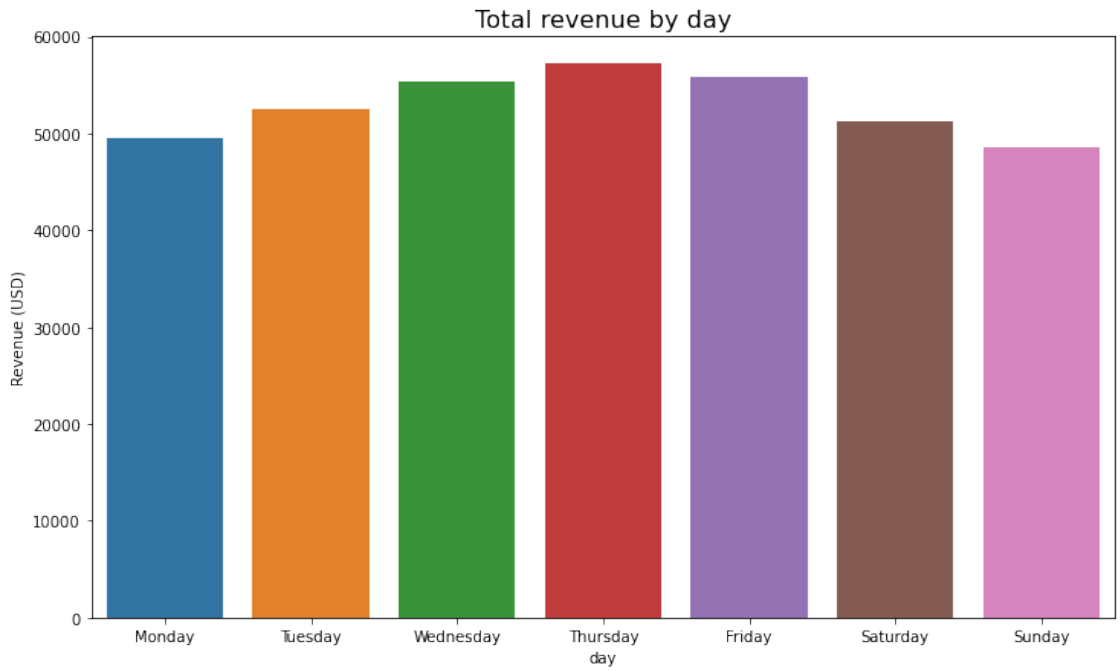
[450]:
```
# Repeat the process, this time for total revenue by day
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday',␣
↪'Saturday', 'Sunday']
total_amount_day = df.groupby('day').sum()[['total_amount']]
total_amount_day = total_amount_day.reindex(index=day_order)
total_amount_day
```

[450]:

	total_amount
day	
Monday	49574.37
Tuesday	52527.14
Wednesday	55310.47
Thursday	57181.91
Friday	55818.74
Saturday	51195.40
Sunday	48624.06

[451]:
```
# Create bar plot of total revenue by day
plt.figure(figsize=(12,7))
ax = sns.barplot(x=total_amount_day.index, y=total_amount_day['total_amount'])
```

```
ax.set_xticklabels(day_order)
ax.set_ylabel('Revenue (USD)')
plt.title('Total revenue by day', fontsize=16);
```



Thursday had the highest gross revenue of all days, and Sunday and Monday had the least. Interestingly, although Saturday had only 35 fewer rides than Thursday, its gross revenue was ~$6,000 less than Thursday's—more than a 10% drop.

Plot total revenue by month

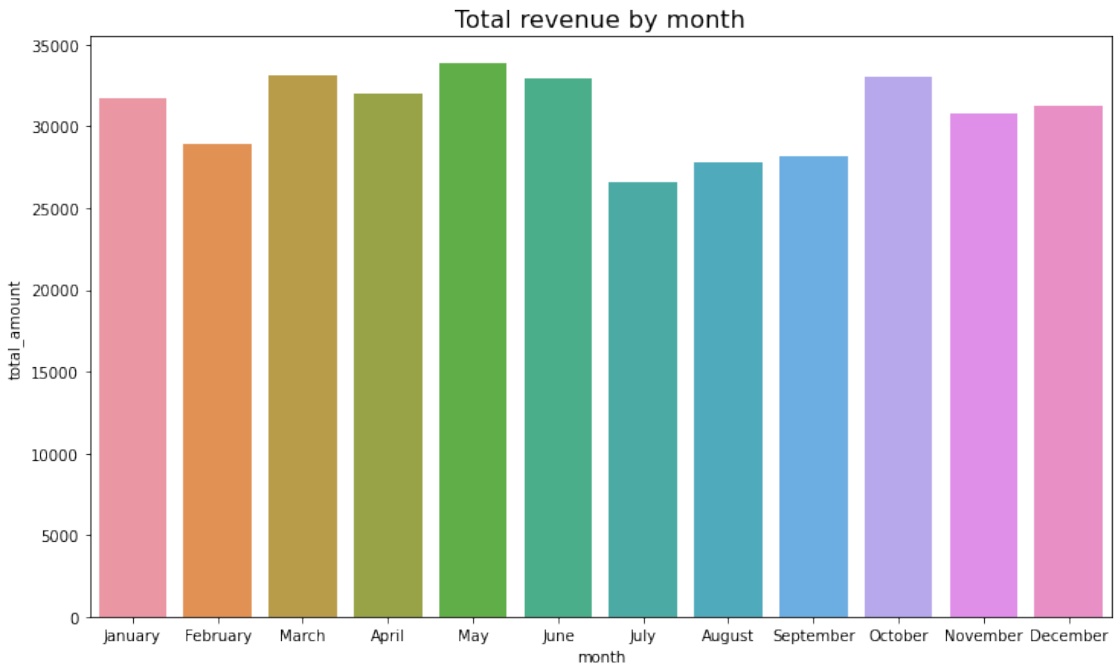
[452]:
```
# Repeat the process, this time for total revenue by month
total_amount_month = df.groupby('month').sum()[['total_amount']]
total_amount_month = total_amount_month.reindex(index=month_order)
total_amount_month
```

[452]:

month	total_amount
January	31735.25
February	28937.89
March	33085.89
April	32012.54
May	33828.58
June	32920.52
July	26617.64
August	27759.56
September	28206.38

```
October        33065.83
November       30800.44
December       31261.57
```

[454]:
```
# Create a bar plot of total revenue by month
plt.figure(figsize=(12,7))
ax = sns.barplot(x=total_amount_month.index,␣
 ↪y=total_amount_month['total_amount'])
plt.title('Total revenue by month', fontsize=16);
```



Monthly revenue generally follows the pattern of monthly rides, with noticeable dips in the summer months of July, August, and September, and also one in February.

You can create a scatterplot in Tableau Public, which can be easier to manipulate and present. If you'd like step by step instructions, you can review the following link. Those instructions create a scatterplot showing the relationship between `total_amount` and `trip_distance`. Consider adding the Tableau visualization to your executive summary, and adding key insights from your findings on those two variables.

Tableau visualization guidelines

Plot mean Trip distance by drop-off location

[455]:
```
# Get number of unique drop-off location IDs
df['DOLocationID'].nunique()
```

[455]: 216

There are **216** unique drop-off zones, which cover virtually the entire mapped area of the New York TLC.

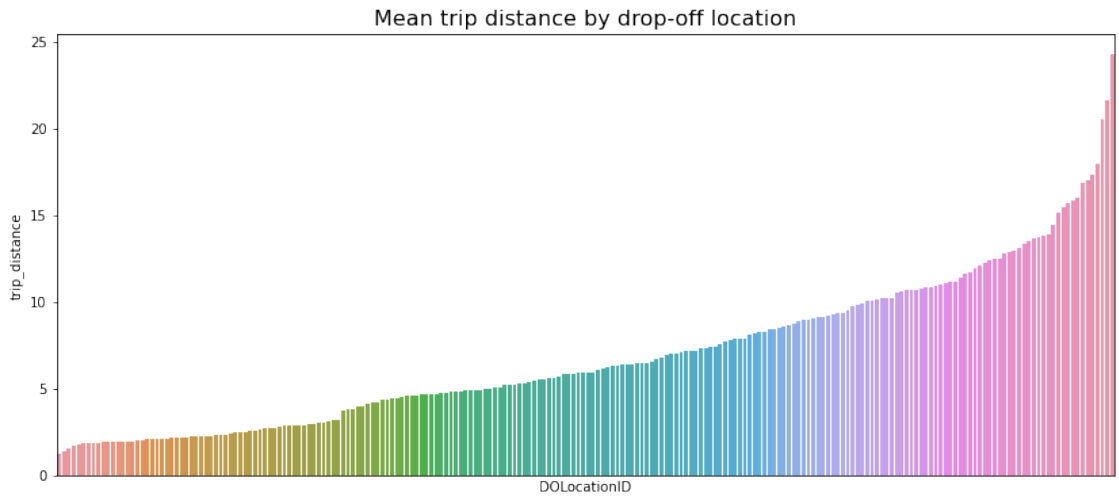
[456]:
```
# Calculate the mean trip distance for each drop-off location
distance_by_dropoff = df.groupby('DOLocationID').mean()[['trip_distance']]

# Sort the results in descending order by mean trip distance
distance_by_dropoff = distance_by_dropoff.sort_values(by='trip_distance')
distance_by_dropoff
```

[456]:
```
              trip_distance
DOLocationID
207                1.200000
193                1.390556
237                1.555494
234                1.727806
137                1.818852
…                     …
51                17.310000
11                17.945000
210               20.500000
29                21.650000
23                24.275000

[216 rows x 1 columns]
```

[457]:
```
# Create a bar plot of mean trip distances by drop-off location in ascending␣
↪order by distance
plt.figure(figsize=(14,6))
ax = sns.barplot(x=distance_by_dropoff.index,
                 y=distance_by_dropoff['trip_distance'],
                 order=distance_by_dropoff.index)
ax.set_xticklabels([])
ax.set_xticks([])
plt.title('Mean trip distance by drop-off location', fontsize=16);
```



This plot presents a characteristic curve related to the cumulative density function of a normal distribution. In other words, it indicates that the drop-off points are relatively evenly distributed over the terrain. This is good to know, because geographic coordinates were not included in this dataset, so there was no obvious way to test for the distibution of locations.

4.4 BONUS CONTENT

To confirm your conclusion, consider the following experiment: 1. Create a sample of coordinates from a normal distribution—in this case 1,500 pairs of points from a normal distribution with a mean of 10 and a standard deviation of 5 2. Calculate the distance between each pair of coordinates 3. Group the coordinates by endpoint and calculate the mean distance between that endpoint and all other points it was paired with 4. Plot the mean distance for each unique endpoint

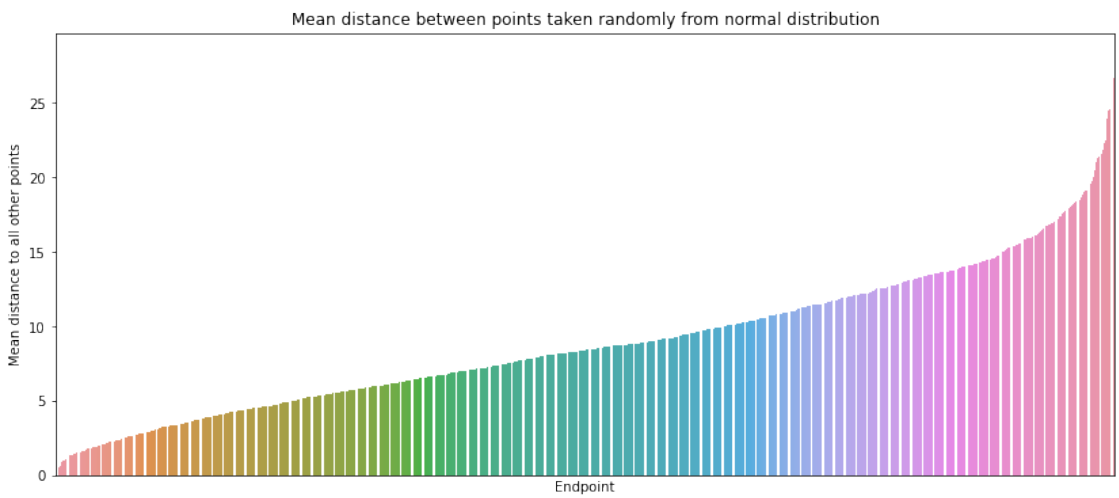
[458]:
```
# 1. Generate random points on a 2D plane from a normal distribution
test = np.round(np.random.normal(10, 5, (3000, 2)), 1)
midway = int(len(test)/2)  # Calculate midpoint of the array of coordinates
start = test[:midway]      # Isolate first half of array ("pick-up locations")
end = test[midway:]        # Isolate second half of array ("drop-off locations")

# 2. Calculate Euclidean distances between points in first half and second half␣
↪of array
distances = (start - end)**2
distances = distances.sum(axis=-1)
distances = np.sqrt(distances)

# 3. Group the coordinates by "drop-off location", compute mean distance
test_df = pd.DataFrame({'start': [tuple(x) for x in start.tolist()],
                    'end': [tuple(x) for x in end.tolist()],
                    'distance': distances})
data = test_df[['end', 'distance']].groupby('end').mean()
```

```
data = data.sort_values(by='distance')

# 4. Plot the mean distance between each endpoint ("drop-off location") and all␣
↪points it connected to
plt.figure(figsize=(14,6))
ax = sns.barplot(x=data.index,
                 y=data['distance'],
                 order=data.index)
ax.set_xticklabels([])
ax.set_xticks([])
ax.set_xlabel('Endpoint')
ax.set_ylabel('Mean distance to all other points')
ax.set_title('Mean distance between points taken randomly from normal␣
↪distribution');
```



The curve described by this graph is nearly identical to that of the mean distance traveled by each taxi ride to each drop-off location. This reveals that the drop-off locations in the taxi dataset are evenly distributed geographically. Note, however, that this does not mean that there was an even distrubtion of rides to each drop-off point. Examine this next.

Histogram of rides by drop-off location

First, check to whether the drop-off locations IDs are consecutively numbered. For instance, does it go 1, 2, 3, 4…, or are some numbers missing (e.g., 1, 3, 4…). If numbers aren't all consecutive, the histogram will look like some locations have very few or no rides when in reality there's no bar because there's no location.

There are many ways to do this.

[459]:
```
# Check if all drop-off locations are consecutively numbered
df['DOLocationID'].max() - len(set(df['DOLocationID']))
```

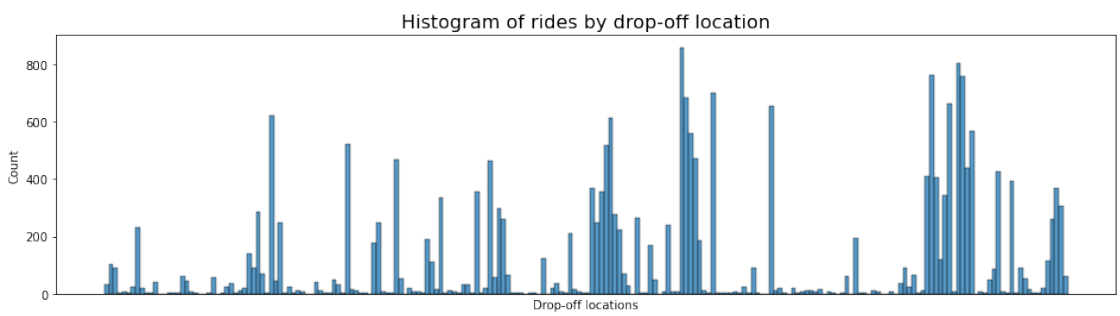
[459]: 49

There are 49 numbers that do not represent a drop-off location.

To eliminate the spaces in the historgram that these missing numbers would create, sort the unique drop-off location values, then convert them to strings. This will make the histplot function display all bars directly next to each other.

To eliminate visual gaps that these missing numbers would create, sort the unique drop-off location values, then convert them to strings. Converting IDs to strings makes the categorical axis render bars directly next to each other in Matplotlib.

[460]:
```
plt.figure(figsize=(16,4))
# DOLocationID column is numeric, so sort in ascending order
sorted_dropoffs = df['DOLocationID'].sort_values()
# Convert to string
sorted_dropoffs = sorted_dropoffs.astype('str')
# Plot
sns.histplot(sorted_dropoffs, bins=range(0, df['DOLocationID'].max()+1, 1))
plt.xticks([])
plt.xlabel('Drop-off locations')
plt.title('Histogram of rides by drop-off location', fontsize=16);
```



Notice that out of the 200+ drop-off locations, a disproportionate number of locations receive the majority of the traffic, while all the rest get relatively few trips. It's likely that these high-traffic locations are near popular tourist attractions like the Empire State Building or Times Square, airports, and train and bus terminals. However, it would be helpful to know the location that each ID corresponds with. Unfortunately, this is not in the data.

4.5 PACE: Execute

Consider the questions in your PACE Strategy Document to reflect on the Execute stage.

4.5.1 Task 4a. Results and evaluation

Having built visualizations in Tableau and in Python, what have you learned about the dataset? What other questions have your visualizations uncovered that you should pursue?

Pro tip: Put yourself in your client's perspective, what would they want to know?

Use the following code fields to pursue any additional EDA based on the visualizations you've already plotted. Also use the space to make sure your visualizations are clean, easily understandable, and accessible.

Ask yourself: Did you consider color, contrast, emphasis, and labeling?

After building visualizations in Tableau and Python, I've gained several insights into the TLC trip dataset:

I have learned that the dataset shows a strong positive correlation between trip distance and total fare, with most trips under 5 miles but a few long-distance outliers.

1. **Trip distance and total fare relationship:**
   The scatter plot shows a strong positive correlation between Trip distance and Total amount. This makes sense, longer trips generally cost more.
   However, I noticed a few records where the Trip distance is `0` but the fare is still positive. These outliers might represent data entry errors or canceled trips that were still charged a base fee.We should apply a data-quality check to remove trips where the distance is zero but the fare is still charged, to prevent these records from distorting our KPIs.

2. **Drop-off location patterns:**
   By calculating the mean Trip distance per drop-off location, I found that some drop-off zones tend to have much longer average trips than others. This may indicate areas that typically receive long-distance rides (e.g., airports or suburbs).
   On the other hand, many drop-off IDs are missing or unused, suggesting parts of the city might have little to no drop-off activity.

3. **Data quality and consistency:**
   The analysis revealed missing or non-consecutive drop-off IDs, meaning not all potential zones are represented in the dataset. This could be important when integrating data across different months or years.

4. **Next questions to pursue:**
   - Are the zero-distance trips concentrated in specific pickup locations or times?
   - Which drop-off zones contribute the most to total revenue?
   - Is there a seasonal or monthly trend in Trip distances or fares?
   - Are there differences in patterns between weekdays and weekends?

5. **Visualization quality and accessibility:**
   I ensured high contrast (WCAG-friendly), large labels, and avoided color-only encoding—important given the TLC stakeholder with visual impairments. In Tableau, color was used to

highlight outlier points (zero-distance trips), and axis ranges were adjusted to make patterns visible. In Python, I added descriptive titles, axes labels, and rotated x-ticks for readability.

From a client's perspective, these findings would be useful to: - Identify potential data issues to correct (e.g., invalid trips).
- Understand which areas of the city bring in the most revenue.
- Guide decisions for optimizing service coverage and pricing strategies.

Overall, these insights will help the client improve data accuracy, identify profitable areas, and develop a more reliable predictive model for trip fares.

[462]:
```
df['trip_duration'] = (df['tpep_dropoff_datetime']-df['tpep_pickup_datetime'])
```

[463]:
```
df.head(10)
```

[463]:
```
   Unnamed: 0  VendorID tpep_pickup_datetime tpep_dropoff_datetime  \
0    24870114         2  2017-03-25 08:55:43   2017-03-25 09:09:47
1    35634249         1  2017-04-11 14:53:28   2017-04-11 15:19:58
2   106203690         1  2017-12-15 07:26:56   2017-12-15 07:34:08
3    38942136         2  2017-05-07 13:17:59   2017-05-07 13:48:14
4    30841670         2  2017-04-15 23:32:20   2017-04-15 23:49:03
5    23345809         2  2017-03-25 20:34:11   2017-03-25 20:42:11
6    37660487         2  2017-05-03 19:04:09   2017-05-03 20:03:47
7    69059411         2  2017-08-15 17:41:06   2017-08-15 18:03:05
8     8433159         2  2017-02-04 16:17:07   2017-02-04 16:29:14
9    95294817         1  2017-11-10 15:20:29   2017-11-10 15:40:55

   passenger_count  trip_distance  RatecodeID store_and_fwd_flag  \
0                6           3.34           1                  N
1                1           1.80           1                  N
2                1           1.00           1                  N
3                1           3.70           1                  N
4                1           4.37           1                  N
5                6           2.30           1                  N
6                1          12.83           1                  N
7                1           2.98           1                  N
8                1           1.20           1                  N
9                1           1.60           1                  N

   PULocationID  DOLocationID  …  extra  mta_tax  tip_amount  tolls_amount  \
0           100           231  …    0.0      0.5        2.76           0.0
1           186            43  …    0.0      0.5        4.00           0.0
2           262           236  …    0.0      0.5        1.45           0.0
3           188            97  …    0.0      0.5        6.39           0.0
4             4           112  …    0.5      0.5        0.00           0.0
5           161           236  …    0.5      0.5        2.06           0.0
6            79           241  …    1.0      0.5        9.86           0.0
7           237           114  …    1.0      0.5        1.78           0.0
```

```
8           234           249  …    0.0      0.5        0.00           0.0
9           239           237  …    0.0      0.5        2.75           0.0

   improvement_surcharge  total_amount  trip_duration_minutes     month  \
0                    0.3         16.56              14.066667     March
1                    0.3         20.80              26.500000     April
2                    0.3          8.75               7.200000  December
3                    0.3         27.69              30.250000       May
4                    0.3         17.80              16.716667     April
5                    0.3         12.36               8.000000     March
6                    0.3         59.16              59.633333       May
7                    0.3         19.58              21.983333    August
8                    0.3          9.80              12.116667  February
9                    0.3         16.55              20.433333  November

         day   trip_duration
0   Saturday 0 days 00:14:04
1    Tuesday 0 days 00:26:30
2     Friday 0 days 00:07:12
3     Sunday 0 days 00:30:15
4   Saturday 0 days 00:16:43
5   Saturday 0 days 00:08:00
6  Wednesday 0 days 00:59:38
7    Tuesday 0 days 00:21:59
8   Saturday 0 days 00:12:07
9     Friday 0 days 00:20:26

[10 rows x 22 columns]
```

4.5.2 Task 4b. Conclusion

Make it professional and presentable

You have visualized the data you need to share with the director now. Remember, the goal of a data visualization is for an audience member to glean the information on the chart in mere seconds.

Questions to ask yourself for reflection: Why is it important to conduct Exploratory Data Analysis? Why are the data visualizations provided in this notebook useful?

Conducting **Exploratory Data Analysis (EDA)** is essential because it helps us understand the structure, quality, and underlying patterns of a dataset before building any models or drawing conclusions. Through EDA, we can identify missing values, outliers, and inconsistencies, and gain insights into how variables relate to each other. It ensures that our later analyses or predictions are based on clean and meaningful data.

The **visualizations in this notebook are useful** because they make complex data easier to interpret. For example, the scatterplots reveal relationships between distance and fare amount, histograms show the distribution of Trip distances, and bar charts highlight average fares across

different ranges. Together, these visuals transform raw numbers into a clear story, allowing us (and stakeholders) to make data-driven decisions with confidence.

You've now completed professional data visualizations according to a business need. Well done!

Congratulations! You've completed this lab. However, you may not notice a green check mark next to this item on Coursera's platform. Please continue your progress regardless of the check mark. Just click on the "save" icon at the top of this notebook to ensure your work has been logged.